Technical Report: Loan Default Risk Prediction with a SHAP Explanation Stability Audit

Henriette Utatsineza (Feza) | BSc (Hons) Software Engineering, African Leadership University | ML track | Supervisor: Elvira Khwatenge | 31 October 2026

Repository: https://github.com/utatsineza/loan-default-risk-audit | Walkthrough video: [ADD LINK]

1. Project summary and scope alignment

The project predicts whether a borrower will become 90 or more days delinquent within two years (Give Me Some Credit, about 150,000 borrowers, 6.7% defaults) and asks a trust question about the explanations: **if two differently configured XGBoost models are equally accurate, do they give the same SHAP reasons for the same borrower?** The system trains cost-sensitive XGBoost models, searches for a matched Model B, and measures the agreement of the two models' SHAP explanations (Spearman rank correlation, top-3 Jaccard index) on 300 audited borrowers. The approved scope had three objectives: (i) handle imbalance with cost-sensitive weighting and compare it with resampling, (ii) build matched-accuracy model pairs, (iii) audit explanation stability. Table 1 maps each planned feature to the repository.

Table 1. Planned features, status and evidence (all files are in the repository).

Planned feature	Intended purpose	Status	Evidence
Preprocessing: median imputation, stratified 80:20 split	Clean data and keep the 6.7% default rate in every split	Completed	`loan_audit/data.py`; 7 tests
Logistic regression baseline	Reference point from the literature	Completed	ROC-AUC 0.802 (`results/metrics.csv`)
Cost-sensitive XGBoost (scale_pos_weight)	Handle imbalance without synthetic data	Completed	ROC-AUC 0.866, PR-AUC 0.400
Cost-sensitive vs SMOTE / ADASYN (Obj. i, RQ1)	Test whether weighting is better than resampling	Completed after supervisor feedback	`resampling_comparison.csv`
Matched-accuracy model pairs (Obj. ii)	Compare explanations of equally accurate models	Completed: 5 pairs, all within tolerance	`pairs_summary.csv`
SHAP explanations	Per-borrower reasons for a prediction	Completed	`explain.py`; app waterfall plot
Stability audit: Spearman, top-3 Jaccard (Obj. iii, RQ2)	Test whether explanations agree	Completed	`audit.py`; `audit_summary.json`
Sensitivity of threshold, top-k and tolerance	Justify the matched and divergent definitions	Completed after supervisor feedback	`sensitivity_*.csv`
Characterise divergent cases (RQ3)	Understand when explanations disagree	Completed, exploratory (15 cases)	`divergence_profile.csv`
Demo interface	Let a user predict and see the explanation	Completed; scope change (see below)	`app.py` (Streamlit)
Automated tests and benchmark	Quality and performance evidence	Added beyond the proposal	62 tests; `benchmark.json`

Scope changes and justification. (1) The interface is a Streamlit web app instead of the planned command line or notebook, because it lets a non-programmer try a borrower and read the explanation; it runs locally and is **not deployed**. (2) After supervisor feedback, the resampling baseline, the multi-pair audit, the explicit definitions of *matched accuracy* and *divergent*, and the sensitivity study were added; the first version used one model pair and had no resampling comparison. (3) Tests and a benchmark were added to support the testing and performance criteria.

2. Technical functionality

Data handling. `DataPreprocessor` validates the schema and the binary target, makes a stratified 80:20 train/test split, then holds out 20% of the training data as a validation set used only for matching (96,000 fit, 24,000 validation, 30,000 test rows). Median imputation for *MonthlyIncome* (about 20% missing) and *NumberOfDependents* (about 2.6%) is fitted on the fit set only and reused for the other partitions, so no test information leaks into training.

Core logic. (a) Model A is an XGBoost classifier with `scale_pos_weight` equal to the non-default to default ratio (about 14). (b) `MatchedModelSelector` trains candidates that differ from Model A in both tree depth and seed, and accepts the first whose validation ROC-AUC is within 0.005 of Model A (the closest candidate is flagged as unmatched otherwise). (c) `AuditSampler` draws 300 test borrowers, half from the borderline band (mean predicted score 0.4 to 0.6) and the rest stratified by true label. (d) `StabilityAuditor` computes SHAP values for both models and scores each borrower; a borrower is **divergent** when the top-3 Jaccard index is below 0.5. (e) `MultiPairAuditor` repeats steps a to d for five pairs with different Model A depths and seeds, and

`DivergenceAnalyzer` compares divergent and agreeing borrowers.

Backend and frontend. The backend (`loan_audit.pipeline`, run from the command line) trains everything and writes its results as CSV, JSON, PNG and model files to `results/`. The Streamlit frontend reads those files: the *Predict and Explain* page loads Model A, scores a borrower entered in the form and draws a SHAP waterfall plot; the *Stability Audit Results* page shows matching, agreement, resampling, pair, sensitivity and divergence results. The two sides are decoupled through files, so the app never retrains.

Inputs and error handling. The pipeline rejects a missing file, a CSV without the required columns, and non-binary or missing targets with clear messages. Models refuse to predict before they are fitted. The form limits every input to a valid range; extreme values (for example 30 late payments) are handled; and the app shows a friendly message instead of a crash when results or models have not been generated yet.

3. Testing and validation

Three strategies are used, run with `python -m pytest`: **unit testing** (50 tests) because the metrics and sampling logic are numerical and easy to get subtly wrong; **integration testing** (6 tests) to check that the stages work together and that every output file is written; and **functional/UI testing** (6 tests) that drive the Streamlit app headlessly through its states. All 62 tests pass in two environments: Windows 11 (Python 3.13.2, 179 s) and Google Colab on Linux (Python 3.13.18, 26 s), whose library versions are much older (pandas 2.2.3 against 3.1.0, scikit-learn 1.6.1 against 1.9.1).

Strategy	Selected because	Examples of cases tested	Result
Unit (50)	Pure functions with known correct answers	Jaccard and Spearman on identical, disjoint and reversed rankings; constant SHAP profiles; imputer uses training medians only; invalid k, sizes and labels; class-weight calculation; resampling balances the training data only; unmatched pairs excluded from the cross-pair summary	All pass
Integration (6)	Stages share data and file formats	Full run writes every output; 1-pair and no-resampling modes; extended outputs match the settings; missing file and malformed CSV fail with clear errors	All pass
Functional / UI (6)	The app is the user's entry point	Prediction page returns a score and class; extreme inputs do not crash; audit page shows the new sections; missing results show an informative message	All pass

Improvements made after testing and review. The exploratory notebook computed imputation medians on the whole dataset before splitting; the package fits them on training data only, and a test now enforces this. The notebook's logistic regression gave a convergence warning on unscaled features; the package scales them first. The same pipeline run on the real data reproduced the audit figures exactly after the refactor into several modules (mean Spearman 0.897, 1.7% divergent), which confirms the restructuring did not change behaviour. XX (`resource` is Unix only); it now has a Windows branch. Running the suite on Colab found a portability defect: with its library versions the matched-model selector returned NumPy booleans and floats instead of plain Python ones, which failed a strict test. The selector now converts them, and all 62 tests pass in both environments.

4. Performance and compatibility

The benchmark (`scripts/benchmark.py`) was run on the full data in two environments: my Windows 11 machine (8 CPU cores, Python 3.13.2) and a Google Colab Linux session (2 CPU cores, Python 3.13.18). The libraries differ widely between them, which doubles as a compatibility check. Results are in `results/benchmark.json` and `results/benchmark_colab.json`.

Measure	Windows 11, 8 cores	Colab Linux, 2 cores
Model fit (120,000 rows)	5.10 s	1.03 s
Single prediction	34.8 ms	1.79 ms
Batch prediction	284,094 rows/s	631,654 rows/s
Single SHAP explanation	78.1 ms	2.41 ms
SHAP for 300 borrowers	0.222 s	0.043 s
Peak memory	249 MB	346 MB
Full test suite (62 tests)	179 s	26 s
scikit-learn / pandas / numpy	1.9.1 / 3.1.0 / 2.5.4	1.6.1 / 2.2.3 / 2.1.3

The same code and tests ran correctly in both environments, across Python 3.13 and a wide range of pandas, scikit-learn and NumPy versions. A prediction with its SHAP explanation takes well under a quarter of a second on either machine, so the app feels immediate, and peak memory stayed below 350 MB. The Colab session, despite having fewer cores, was 2 to 32 times faster on most measures. I did not isolate the cause (operating system, library versions, processor generation and background load on my machine are all possible), so the table shows that performance is adequate in both, not a ranking of platforms. Peak memory is read from different

operating-system counters (peak working set on Windows, maximum resident set on Linux), so those two figures are not strictly comparable. **Limitations:** two environments only; no low-end physical device was tested, and the app was viewed in one browser.

5. Key results

RQ1: cost-sensitive weighting vs resampling. Same XGBoost settings and test set; only the handling of imbalance differs.

Strategy	PR-AUC	ROC-AUC	Best F1	Recall at 0.5	Precision at 0.5
Cost-sensitive (Model A)	0.400	0.866	0.443	0.760	0.223
No imbalance handling	0.401	0.868	0.447	0.201	0.587
SMOTE	0.302	0.833	0.368	0.515	0.275
ADASYN	0.298	0.831	0.363	0.517	0.274

SMOTE and ADASYN are clearly worse on the threshold-free metrics (about 0.10 lower PR-AUC), which supports the choice of cost-sensitive weighting over resampling. Weighting is no better than doing nothing for ranking quality; its effect is on the decision threshold (recall 0.76 against 0.20 at 0.5). This is one split and one seed, without confidence intervals.

RQ2: are explanations stable? All five matched pairs met the 0.005 tolerance. Across pairs the mean Spearman correlation is 0.89 to 0.91 (mean 0.90), mean top-3 Jaccard 0.84 to 0.87, the top feature is the same for 91 to 94% of borrowers, and 0.0% to 1.7% of cases are divergent (mean 1.0%). The headline depends strongly on the definition, however:

Top-k compared	Jaccard below 0.3	below 0.5 (adopted for k=3)	below 0.7	Any difference (below 1.0)
k = 2	0.0%	23.0%	23.0%	23.0%
k = 3	1.7%	1.7%	27.7%	27.7%
k = 5	0.0%	3.7%	55.7%	55.7%

The two models almost always agree on the main driver, but 28% of borrowers get at least one different feature in their top 3 and 56% in their top 5, so the 1.7% divergence figure should be read together with this table. Matching Model B more tightly (tolerance 0.002) gave slightly higher agreement (Spearman 0.909, 0.7% divergent); the 0.01 setting gave the same pair as 0.005 because the first candidate already matched.

RQ3: who diverges? Pooling all pairs gives 15 divergent and 1,485 agreeing cases. Divergent borrowers are younger (median age 32 against 48, Mann-Whitney p = 0.019, not corrected for ten tests) and are not concentrated in the borderline score band (47% against 57%). The features most often in one model's top 3 but not the other's are the 30 to 59 days past due count (14 cases), the 90+ days late count (12) and the debt ratio (10); overlapping delinquency features may let two models share credit differently, which is a hypothesis for future work and not a finding.

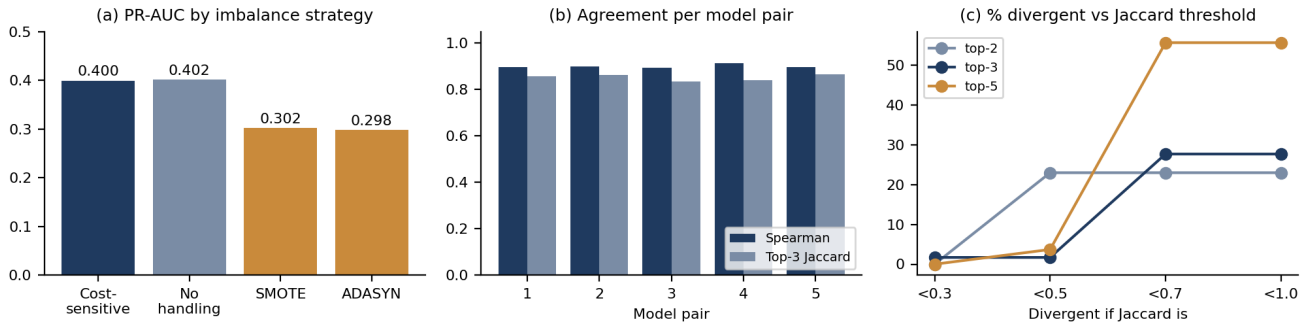


Figure 1. (a) PR-AUC for each imbalance strategy; (b) mean agreement of the five matched model pairs; (c) share of audited borrowers flagged as divergent for different top-k sizes and thresholds.

6. Code quality and engineering practices

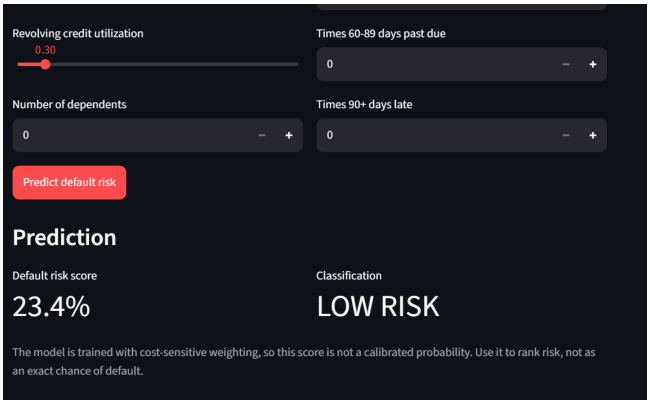
- **Abstraction and polymorphism:** `BaseModel` defines `fit` and `predict_proba`; `LogisticBaseline` and `XGBoostModel` implement it, so the pipeline treats every model alike (open for new models, closed for modification).
- **Encapsulation:** the preprocessor keeps its learned medians private and refuses to transform before fitting; XGBoost guards access to an unfitted model.
- **Separation of concerns and single responsibility:** data, models, resampling, metrics, explanation, sampling, auditing, multi-pair orchestration, divergence analysis, plotting and the pipeline each have their own module; `Config` holds every threshold, so there are no magic numbers.

- **Reuse and immutability:** one `PairExperiment` class serves the main pair, the extra pairs and the tolerance sweep; results are immutable dataclasses (`SHAPResult`, `AgreementScore`, `PairResult`).
- **Algorithms and problem solving:** cost-sensitive weighting; first-fit matched-accuracy search over a candidate grid using validation data only; borderline-weighted stratified audit sampling without duplicates; Spearman and top-k Jaccard agreement with explicit handling of constant profiles; a sensitivity grid that reuses SHAP values; best-F1 over all thresholds for fair comparison across imbalance strategies.
- **Readability and organisation:** consistent naming, docstrings that state each algorithm, a README with installation, usage, outputs, definitions and repository structure, and an exploratory notebook kept apart from the code of record.

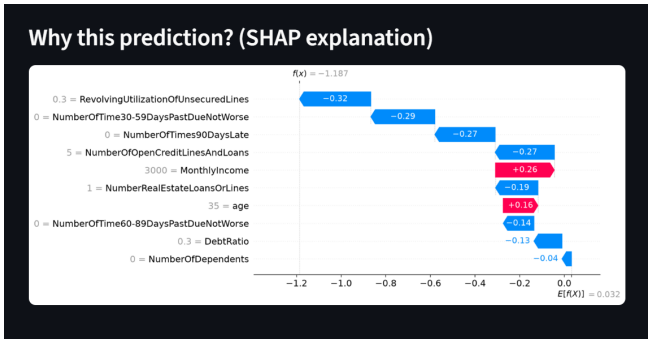
7. Limitations and conclusion

This is a research prototype, not a lending system. The data are a 2011 competition dataset with no documented Rwandan origin, so the findings are not claimed to generalise to Rwandan borrowers; scores are not calibrated probabilities; the age finding in RQ3 is exploratory and relevant to fairness; and the divergence figure depends on analyst choices, which Section 5 quantifies. The main conclusion is that two equally accurate models give highly consistent explanations of the *main* risk driver but often differ on the second and third, so SHAP rankings beyond the top feature should be treated with caution. Remaining work includes testing on more hardware and browsers, repeating the study with several data splits, and testing the delinquency-feature hypothesis.

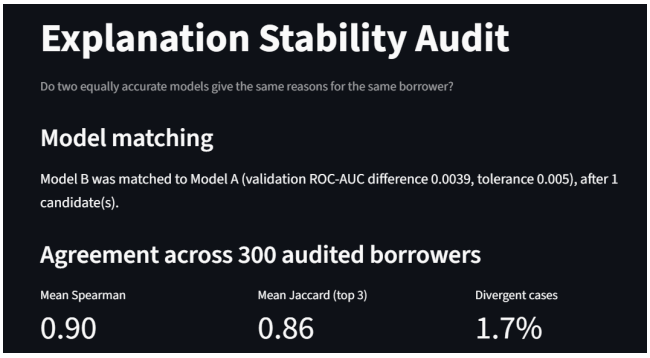
Evidence: the running application



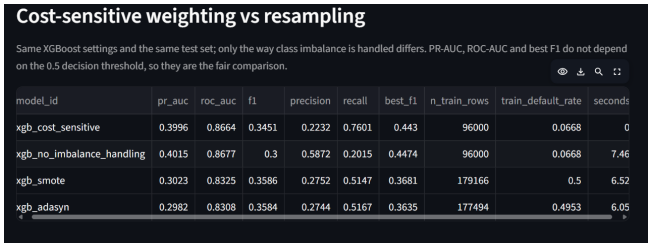
(a) Prediction page: score and class



(b) SHAP waterfall for the same borrower



(c) Audit page: headline agreement figures



(d) Cost-sensitive vs resampling table

Figure 2. Screenshots of the Streamlit app running on the real data (Windows 11).